Binary-Search

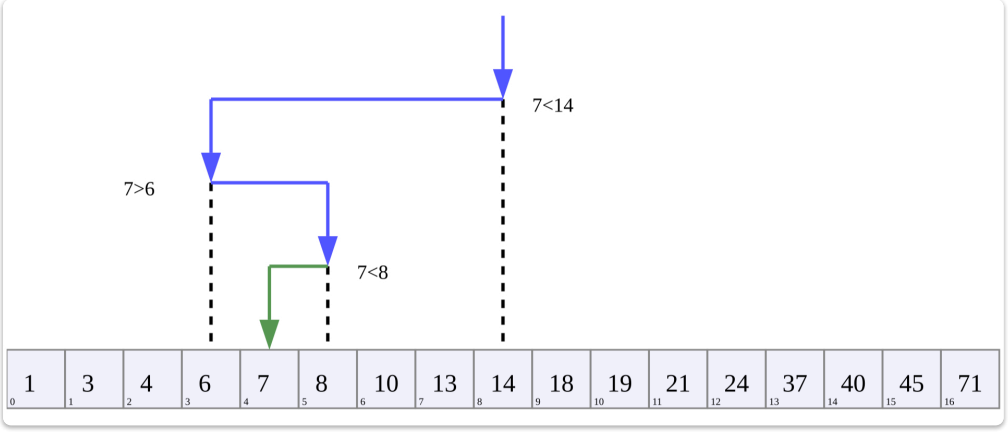
Binary Search (Búsqueda Binaria / Dicotómica)

La búsqueda binaria es un algoritmo de búsqueda eficiente que localiza un elemento dentro de un array ordenado (supondremos orden creciente).

En lugar de recorrer todos los elementos uno a uno, aplica la idea de divide y vencerás:

1. Mira el elemento central del rango actual.
2. Lo compara con el valor buscado:
   - Si son iguales → encontrado.
   - Si el valor buscado es menor → la búsqueda continúa en la mitad izquierda.
   - Si es mayor → la búsqueda continúa en la mitad derecha.
3. Repite el proceso sobre el nuevo rango, reduciendo el espacio de búsqueda a la mitad en cada paso.

> *Condición esencial: el array debe estar ordenado.*
> *Si no lo está, la búsqueda binaria no funciona correctamente.*



Complejidad temporal:

- Mejor caso: (O(1)) (lo encontramos en la primera comparación).
- Caso medio y peor caso: (O(\log_2 n)).

Complejidad espacial: (O(1)) (solo usa unas pocas variables).

2. Esquema paso a paso (explicación detallada)

Supongamos un array ordenado `A` de longitud `n` y un valor a buscar `x`.

1. **Definir el rango inicial de búsqueda**
   - `inicio = 0`
   - `fin = n - 1`
     El valor buscado se encuentra, si existe, en el rango de índices `[inicio..fin]`.
2. **Mientras el rango siga siendo válido**
   - Condición: `inicio <= fin`.
   - Mientras esto se cumpla, todavía hay elementos que comprobar.
3. **Calcular la posición central del rango**
   - Fórmula típica:
     ```
     centro = (inicio + fin) // 2      # división entera
     ```
   - En muchos lenguajes (y para evitar overflow con enteros grandes) se recomienda:
     ```
     centro = inicio + (fin - inicio) // 2
     ```
   - `A[centro]` es el elemento central del rango actual.
4. **Comparar el elemento central con el valor buscado `x`**
   - Si `A[centro] == x`
     → **Éxito**: se ha encontrado el valor. Devolvemos `centro`.
   - Si `x < A[centro]`
     → El valor buscado, si está, debe estar a la **izquierda**
     → Actualizamos `fin = centro - 1` (reducimos el rango a la mitad izquierda).
   - Si `x > A[centro]`
     → El valor buscado debe estar a la **derecha**
     → Actualizamos `inicio = centro + 1` (reducimos el rango a la mitad derecha).
5. **Repetir**

- Volvemos al paso 2 con los nuevos `inicio` y `fin`.
- En cada iteración, el tamaño del rango se reduce aproximadamente a la mitad.

6. Si el rango se agota sin encontrar el valor
   - Es decir, en algún momento `inicio > fin`.
   - Conclusión: el valor no está en el array.
   - Devolvemos típicamente `-1` para indicar que no se ha encontrado.

Mini ejemplo

Array ordenado:
`A = [4, 5, 9, 12, 14, 15, 20, 22]`
Valor buscado: `x = 15`

- Iteración 1:
  - `inicio = 0`, `fin = 7`
  - `centro = (0 + 7) // 2 = 3`
  - `A[3] = 12`
  - Como `15 > 12`, buscamos en la derecha → `inicio = centro + 1 = 4`.
- **Iteración 2:*
  - `inicio = 4`, `fin = 7`
  - `centro = (4 + 7) // 2 = 5`
  - `A[5] = 15`
  - `A[centro] == x` → encontrado en la posición `5`.

---

3. Pseudocódigo (estilo CLRS, índices desde 0)

Versión iterativa, que devuelve el índice del elemento buscado o `-1` si no existe:

```
BINARY-SEARCH(A, x)
    n = length(A)
    inicio = 0
```

```
    fin = n - 1

    while inicio <= fin
        centro = inicio + (fin - inicio) // 2
        if A[centro] == x
            return centro
        else if x < A[centro]
            fin = centro - 1
        else
            inicio = centro + 1
    end while

    return -1
```

---

4. Implementación en Python

```python
def binary_search(arr, x):
    """
    Búsqueda binaria iterativa.
    Devuelve el índice de x si se encuentra en arr (ordenado),
    o -1 en caso contrario.
    """
    inicio = 0
    fin = len(arr) - 1

    while inicio <= fin:
        # Usamos esta forma para evitar overflow en otros lenguajes,
        # aunque en Python no es un problema práctico.
        centro = inicio + (fin - inicio) // 2
        valor_central = arr[centro]

        if valor_central == x:
            return centro
        elif x < valor_central:
            fin = centro - 1
        else:
            inicio = centro + 1
```

```python

    return -1


if __name__ == "__main__":
    array = [4, 5, 9, 12, 14, 15, 20, 22]
    x = 15
    pos = binary_search(array, x)

    if pos != -1:
        print(f"Valor {x} encontrado en la posición {pos}")
    else:
        print(f"Valor {x} no encontrado en el array")
```